

TD2

Étude de cas : `minibuild` — Gestionnaire de dépendances

M2 MIAGE CTAQ / TD2

TDD · Tests unitaires · Tests d'intégration · Tests de validation

Version 1.0-alpha

Partie 1 — Présentation du sujet

Objectifs

Dans ce TD vous allez construire un **mini-gestionnaire de dépendances**, dans l'esprit de Maven ou Gradle, mais réduit à l'essentiel. Un fichier de build déclare des dépendances ; votre outil doit les lire, interroger un registre pour les retrouver, et résoudre les dépendances transitives.

Ce système est **architecturé en couches**, du parsing de bas niveau jusqu'à l'orchestration de haut niveau. Cela va vous permettre de pratiquer les trois niveaux de test du cycle en V :

- **Tests unitaires** — chaque composant isolé de ses collaborateurs, remplacés par des doublures. Vous verrez la différence entre vérifier un *état* et vérifier une *interaction*.
- **Tests d'intégration** — assembler progressivement les composants, en stratégie descendante puis ascendante, et mesurer ce que chaque ordre coûte en pilotes et en bouchons.
- **Tests de validation** — tout câblé, un vrai fichier en entrée, un vrai résultat en sortie.

Le code sera construit en **TDD** : on écrit d'abord un test qui échoue, puis le code minimal qui le fait passer, avant de remanier le code (cycle *red-green-refactor*).

*Vous pouvez utiliser l'IDE de votre choix, **mais** l'ensemble du travail doit pouvoir être construit et exécuté hors IDE avec `./gradlew`.*

Abréviations et conventions

GAV — **coordonnée** d'un artefact, formée de son groupe, de son nom et de sa version
(`group:artifact:version`) ; le terme vient de Maven (*Maven coordinates*, maven.apache.org/pom.html)
SUT — *System Under Test*, composant testé
pilote — *driver*, simule un appelant (cours C2)

bouchon — *stub*, simule un appelé (cours C2)
doublure — *test double*, remplace un collaborateur dans un test (partie 3, cours Outils)
TU — test unitaire
TI — test d'intégration
TV — test de validation

Le système à construire

Les règles métier de `minibuild` données dans ce sujet (format du fichier de build, non-republication, résolution sans gestion des conflits de versions) valent pour ce TD. D'autres sujets pourront les faire évoluer, par exemple en retenant comme Maven la version la plus proche du projet.

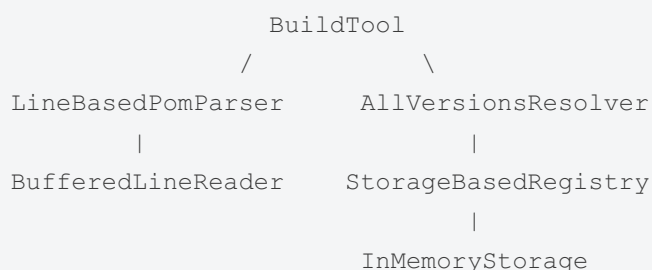
Le système `minibuild` est composé de six éléments organisés en couches :

Interface	Réalisation	Rôle
—	<code>BuildTool</code>	Orchestre : lit le fichier de build, puis résout les dépendances
<code>IResolver</code>	<code>AllVersionsResolver</code>	Calcule la fermeture transitive des dépendances, en interrogeant le registre pour connaître celles de chaque artefact
<code>IRegistry</code>	<code>StorageBasedRegistry</code>	Publie et recherche les artefacts ; refuse de republier une coordonnée déjà publiée
<code>IPomParser</code>	<code>LineBasedPomParser</code>	Parse un fichier de build → dépendances déclarées
<code>IStorage</code>	<code>InMemoryStorage</code>	Associe à chaque coordonnée l'artefact correspondant (stockage clé-valeur)
<code>ILineReader</code>	<code>BufferedReader</code>	Lecture ligne à ligne (adaptateur sur <code>BufferedReader</code>)

Comme sur Maven Central, un artefact publié ne peut plus être modifié : pour corriger une erreur, il faut publier une nouvelle version (central.sonatype.org/faq/can-i-change-a-component).

Chaque composant (sauf `BuildTool`, que rien n'appelle) réalise une interface `IXxx`. Ses appelants ne dépendent que de cette interface : on peut ainsi lui substituer une autre réalisation, en particulier une doublure. `BuildTool` n'a pas d'interface, car aucun composant n'en dépend : une abstraction sans client n'aurait pas d'utilité. Le nom de chaque réalisation dit ce qui la distingue d'une autre réalisation possible de la même interface (par exemple, `AllVersionsResolver` garde toutes les versions en conflit, voir plus bas).

Le **graphe d'appel** entre composants est le suivant :



Ce graphe d'appel est l'élément central de la partie sur les tests d'intégration : c'est lui qui détermine les ordres d'intégration possibles, ainsi que le nombre de pilotes et de bouchons nécessaires à chaque étape.

Format du fichier de build

Le fichier de build, équivalent très simplifié du `pom.xml` de Maven, décrit le projet et ses dépendances **directes**, à raison d'une déclaration par ligne :

```
project mon-app
dependency org.acme:lib-a:1.0.0
dependency org.acme:lib-b:2.1.0
```

La première ligne déclare le nom du projet. Chaque ligne suivante déclare une dépendance sous forme de coordonnée `group:artifact:version`. Les lignes vides sont ignorées.

Dépendances transitives. Un artefact publié déclare lui-même ses dépendances. Si `mon-app` dépend de `org.acme:lib-a:1.0.0`, et que `lib-a:1.0.0` dépend à son tour de `org.other:lib-c:3.0.0`, alors `lib-c:3.0.0` est une dépendance **transitive** (indirecte) de `mon-app`. Elle n'apparaît pas dans le fichier de build, mais le résolveur doit la trouver :

```
mon-app —> org.acme:lib-a:1.0.0 —> org.other:lib-c:3.0.0
      |> org.acme:lib-b:2.1.0
```

Le résultat de la résolution est la **fermeture transitive** des dépendances déclarées : ici `{lib-a:1.0.0, lib-b:2.1.0, lib-c:3.0.0}`.

Conflits de versions. Si deux versions d'un même artefact apparaissent dans la fermeture, `minibuild` les garde **toutes les deux** : il ne résout pas les conflits. C'est une simplification volontaire. Maven retient la version la plus proche du projet dans l'arbre des dépendances (« nearest definition », maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html) ; Gradle retient par défaut la plus haute (docs.gradle.org/current/userguide/graph_resolution.html).

Types de données

Les types `Gav`, `Artifact` et `Project` sont à construire au fil du TD. `Gav` porte une vraie logique (découpage et validation d'une chaîne) : il sera construit en TDD. Un `Artifact` est ce qui est publié dans le registre : sa coordonnée et l'ensemble des coordonnées dont il dépend **directement** ; ses dépendances transitives s'obtiennent en suivant celles-ci. Un `Project` est ce que décrit le fichier de build : un nom et l'ensemble de ses dépendances **directes**. Ce sont deux types distincts : un projet n'a pas de coordonnée et n'est pas publié. `Artifact` et `Project` sont de simples agrégats de données. On les écrira sous forme de `record` Java au moment où on en aura besoin, sans les tester en tant que tels : `equals`, `hashCode` et les accesseurs sont engendrés par le langage, et les tester reviendrait à tester Java.

Interfaces fournies

```
public interface ILineReader {
    String readLine() throws IOException;
}

public interface IStorage {
    void put(Gav gav, Artifact artifact);
    Optional<Artifact> get(Gav gav);
}

public interface IRegistry {
    // lève AlreadyPublishedException si la coordonnée de l'artefact est déjà
    // publiée
    void publish(Artifact artifact) throws AlreadyPublishedException;
    Optional<Artifact> lookup(Gav gav);
}

public interface IResolver {
    Set<Gav> resolve(Set<Gav> directDependencies);
}

public interface IPomParser {
    Project parse(ILineReader reader) throws IOException;
}
```

Partie 2 — TDD : les premiers pas

Dans cette partie, on ne construit que des composants qui **n'appellent aucun autre composant** : aucune doublure n'est donc nécessaire. Les composants qui en appellent d'autres sont l'objet de la partie 3.

Mise en place

Question 1. Créer un projet Gradle et le versionner sur GitHub. Ajouter les dépendances de test : JUnit 5, Hamcrest, Mockito et le plugin JaCoCo. Vérifier que tout fonctionne avec `./gradlew test`.

Question 2. À quoi servent respectivement JUnit, Hamcrest, Mockito et JaCoCo ? Lequel n'est pas une bibliothèque de test à proprement parler ?

Réponse :

La task list

Nous allons tenir à jour une *task list* des fonctionnalités à réaliser. À chaque étape, nous choisissons une tâche, et nous en ajoutons éventuellement de nouvelles découvertes en chemin.

```
[X] 1- description de la tâche
[ ] 2- description de la tâche
...
```

Question 3. Construire la task list initiale du projet `minibuild` à partir de la description du système ci-dessus.

Réponse :

Fake it !

Un cycle TDD a trois temps : **red** (écrire un test qui échoue), **green** (écrire le code minimal qui le fait passer), **refactor** (remanier le code sans changer son comportement : les tests restent verts). Le choix de la première tâche est important : elle doit être assez simple pour boucler un premier cycle *red-green-refactor* rapidement. Ici la brique la plus élémentaire est la coordonnée `Gav`.

Question 4. Créer une classe de tests `GavTest`. Y écrire, avec les seules assertions de JUnit (`assertEquals`), un test qui construit une coordonnée à partir de la chaîne `"org.acme:lib-a:1.0.0"` et vérifie que le groupe vaut `"org.acme"`. Vérifier que ce test **ne compile pas** ou **échoue** — la *red bar* est le point de départ obligatoire.

Question 5. Appliquer la technique *Fake it* : écrire le minimum nécessaire pour atteindre la *green bar*, c'est-à-dire une méthode `parse` qui renvoie une valeur constante. Comparer le test et le code : quelle information apparaît dans les deux ? Pourquoi est-ce le signe qu'il reste du travail ?

Réponse :

Triangulation

La méthode *Fake it* montre qu'on peut faire passer un test sans écrire le code correspondant. Le problème vient d'un test insuffisamment spécifié. **Triangler** consiste à raffiner le test pour cerner le comportement attendu.

Question 6. Ajouter un second test, sur la coordonnée `"org.other:lib-c:3.0.0"`, en vérifiant le groupe, l'artefact et la version. Faire ensuite évoluer `Gav.parse` pour qu'elle découpe réellement la chaîne.

Question 7. Au sens du TD1, ces deux coordonnées appartiennent-elles à la même classe d'équivalence ? Pourquoi la triangulation en demande-t-elle quand même deux ?

Réponse :

Question 8. Vos deux tests ne diffèrent que par leurs données. Les regrouper en un seul **test paramétré** JUnit (`@ParameterizedTest`), dont chaque jeu de valeurs donne une entrée et les sorties attendues. Fournir ces jeux de valeurs d'abord dans le code (valeurs CSV écrites dans l'annotation `@CsvSource`, ou méthode `@MethodSource` qui renvoie par exemple un `Object[][]`), puis dans un fichier CSV placé dans les ressources de test (`src/test/resources`, `@CsvFileSource`).

Question 9. Une coordonnée mal formée doit lever une exception. Déterminer, comme au TD1, les classes d'équivalence invalides de l'entrée de `Gav.parse`, puis en déduire les tests, écrits *avant* le code avec `assertThrows` (JUnit). Ajouter les cas manquants à la task list.

Factoriser la mise en place du contexte

Le contexte dans lequel s'exécute un test s'appelle sa *fixture*. Quand plusieurs tests partagent la même, JUnit 5 permet de la construire avant chaque test (méthode annotée `@BeforeEach`) :

```
private IStorage storage;    // typé par l'interface

@BeforeEach
void init() {
    storage = new InMemoryStorage();
}
```

Question 10. En TDD sur `InMemoryStorage`, écrire les tests puis le code pour `put` et `get`, avec une fixture commune. La signature fournie de `get` renvoie un `Optional` : pourquoi est-ce préférable à `null` ou à une exception pour une coordonnée absente ?

Réponse :

BufferedReader

Question 11. Écrire la classe `BufferedReader`, qui réalise `LineNumberReader` en déléguant à un `BufferedReader`, et la tester sur un `StringReader` : aucun fichier n'est nécessaire. Le TDD vous a-t-il aidé sur ce composant ? Pourquoi ?

Réponse :

Hamcrest

Hamcrest (rappel du cours Outils) fournit des *matchers* expressifs et composables :

```
assertThat(gav.group(), is("org.acme"));
assertThat(gav.toString(), startsWith("org.acme"));
assertThat(gav.version(), not(emptyString()));
```

D'autres *matchers* portent sur les collections (`containsInAnyOrder`, `hasSize`, `hasItem`) : on les utilisera en partie 3.

Question 12. Réécrire les assertions de `GavTest` avec Hamcrest. Faire échouer volontairement une assertion dans chacune des deux versions (`assertEquals` et `assertThat`) et comparer les messages d'échec.

Réponse :

Partie 3 — Tests unitaires

Un test est **unitaire** lorsqu'il exerce **un seul composant**, tous ses collaborateurs étant remplacés par des doublures. Sans cela, l'échec d'un test ne permet pas de localiser le défaut, et l'activité d'intégration se trouve détournée de son objectif — vérifier les **interfaces** entre composants. Pour pouvoir substituer une doublure à un collaborateur, chaque composant reçoit ses collaborateurs **par son constructeur** (injection de dépendances). Grâce aux interfaces et à l'injection, une doublure peut s'écrire **à la main** : une classe de test qui implémente l'interface. Mockito ne fait qu'automatiser ce travail. Le cours le dit déjà (C2, diapositive 25) : plus la conception respecte forte cohésion et faible couplage, plus l'écriture des pilotes et des bouchons est simple.

Doublures de test

Le cours emploie le vocabulaire **pilote** / **bouchon** (C2, diapositives 24-25). La littérature du test unitaire utilise une taxonomie plus fine, due à Meszaros (*xUnit Test Patterns*, 2007), qui regroupe

toutes ces substitutions sous le terme générique de **doublure** (*test double*, par analogie avec la doublure de cinéma ; rappel du cours Outils) :

Doublure	Définition
Dummy	passée en paramètre mais jamais utilisée (et ne devrait jamais l'être) ; remplit une signature
Fake	implémentation qui fonctionne, mais avec un raccourci la rendant impropre à la production
Stub (bouchon)	fournit des réponses en dur ; ne vérifie rien
Spy	bouchon qui enregistre en plus les appels reçus ; vérification après l'exécution
Mock	attentes programmées à l'avance ; échoue si le protocole n'est pas respecté

Deux lectures courtes de Martin Fowler pour approfondir : **TestDouble** (martinfowler.com/bliki/TestDouble.html), la taxonomie en une page ; et **Mocks Aren't Stubs** (martinfowler.com/articles/mocksArentStubs.html), sur la distinction entre vérification d'état et vérification d'interaction.

Attention : l'API de Mockito ne distingue pas ces types. Un même objet produit par `mock(...)` joue le rôle d'un stub lorsqu'on programme ses réponses avec `when`, et celui d'un **spy** lorsqu'on vérifie après coup ses appels avec `verify` : c'est le test qui vérifie, après l'exécution. Mockito ne fournit pas de mock au sens de Meszaros (attentes déclarées avant l'exécution, vérifiées par la doublure elle-même) : d'autres outils le font (voir le cours Outils). Dans ce TD, on n'utilise que Mockito.

Décrire un test avant de l'écrire

Avant chaque test de cette partie, on en écrit une description courte, indépendante du code et de Mockito :

$$D_1, \dots, D_n \vdash \text{appel} \Rightarrow \text{attendu}$$

- À gauche de $\vdash$, les doublures, qui forment le contexte du test. Chacune est notée $[Interface \mid comportement]k$, où k donne le type de doublure : d (dummy), f (fake), s (stub), sp (spy), m (mock).
- Dans un comportement, $m(x) \mapsto v$ signifie que l'appel $m(x)$ renvoie v ; $m \mapsto \langle v_1, v_2, \dots \rangle$ signifie que les appels successifs renvoient v_1, v_2 , etc. ; $\perp$ désigne `null`. Pour un spy ou un mock, $\times n$ signifie que l'appel doit avoir lieu exactement n fois.
- À droite, l'appel au SUT et ce qui est attendu : une valeur, ou une exception $\downarrow E$.
- Les lettres en italique désignent des valeurs quelconques. $\{a, b\}$ est un ensemble (en extension), $\langle v_1, v_2 \rangle$ une séquence. On peut quantifier : $\forall g \in \{a, b\} \cdot \text{lookup}(g) \times 1$.

Exemple (question 14.3) :

```
[ILineReader | readLine  $\mapsto$  {"project mon-app",
                                "dependency org.acme:lib-a:1.0.0",  $\perp$ }]s
 $\vdash$  parse(reader)  $\Rightarrow$  Project("mon-app", {org.acme:lib-a:1.0.0})
```

Bien que créée avec Mockito, cette doublure est de type **s** : elle ne fait que fournir des réponses.

Doublures avec Mockito

Mockito permet deux usages distincts d'une doublure. Exemple sur `AllVersionsResolver`, dans le cas le plus simple (un artefact sans dépendance) :

```
[IRegistry | lookup(a) ↦ A]s ⊢ resolve({a}) ⇒ {a}      où A = Artifact(a, ∅)
```

```
// lib-a n'a pas de dépendance
Gav a = Gav.parse("org.acme:lib-a:1.0.0");
Artifact artifactA = new Artifact(a, Set.of());

// Stub : on programme les réponses
// [IRegistry | lookup(a) ↦ A]s
IRegistry registry = mock(IRegistry.class);
when(registry.lookup(a)).thenReturn(Optional.of(artifactA));

// La doublure est injectée dans le SUT, puis on appelle le SUT
IResolver resolver = new AllVersionsResolver(registry);
Set<Gav> resolved = resolver.resolve(Set.of(a));

// Spy : on vérifie après coup les interactions
verify(registry).lookup(a);           // équivaut à times(1)
verify(registry, never()).publish(any());
```

Question 13. Dans le cours (C2, diapositive 25), un *bouchon* « simule la logique de comportement » d'un composant appelé. En quoi la vérification par `verify` va-t-elle au-delà de ce rôle ? Formuler la différence entre test orienté **état** et test orienté **interaction**.

Réponse :

Test unitaire de `LineBasedPomParser`

`LineBasedPomParser` dépend de `ILineNumberReader`. Le remplacer par une doublure (ici un bouchon, créé avec Mockito) évite toute dépendance au système de fichiers.

```
ILineNumberReader reader = mock(ILineNumberReader.class);
when(reader.readLine()).thenReturn(
    "project mon-app",
    "dependency org.acme:lib-a:1.0.0",
    null);
Project project = new LineBasedPomParser().parse(reader); // appel au SUT
```

Question 14. Construire `LineBasedPomParser.parse` par cycles TDD successifs, en faisant passer les cas suivants un par un. Pour chaque cas, commencer par écrire la description du test. Ce sont des tests orientés **état** : on vérifie le `Project` renvoyé.

- **14.1.** fichier vide ;
- **14.2.** fichier ne contenant que la ligne `project` ;
- **14.3.** fichier avec une dépendance ;
- **14.4.** fichier avec plusieurs dépendances et des lignes vides ;
- **14.5.** fichier mal formé (ligne inconnue, coordonnée invalide).

Question 15. La doublure de `ILineReader` n'ouvre aucun fichier. Quels défauts ce test unitaire est-il par construction **incapable** de détecter ?

Réponse :

Test unitaire de `StorageBasedRegistry`

`StorageBasedRegistry` s'appuie sur `IStorage`. Sa responsabilité propre est la règle de non-republication (voir partie 1).

Question 16. En TDD, avec un bouchon de `IStorage`, construire `publish` et `lookup`. Écrire d'abord ce bouchon à la main (une classe qui implémente `IStorage`), puis avec Mockito, et comparer. Écrire en particulier le test de la règle de non-republication : lorsque la coordonnée est déjà présente dans le stockage, `publish` doit lever une exception. Ce test vérifie-t-il un état, une interaction, ou les deux ? Commencer par écrire la description du test.

Réponse :

Test unitaire de `AllVersionsResolver` — état et interaction

`AllVersionsResolver` interroge un `IRegistry` pour connaître les dépendances de chaque artefact, puis calcule la fermeture transitive. Les deux tests qui suivent restent des tests **unitaires** : seul `AllVersionsResolver` est réel, le registre est une doublure. Ce qui change, c'est ce qu'on vérifie : un état (le résultat) ou une interaction (les appels au registre).

Question 17. Écrire un test **orienté état** : avec un registre bouchonné où `org.acme:lib-a:1.0.0` dépend de `org.acme:lib-b:2.1.0`, qui dépend de `org.other:lib-c:3.0.0`, vérifier que `resolve({lib-a})` renvoie bien les trois coordonnées : d'abord avec `assertEquals(Set.of(...), ...)`, puis avec le matcher Hamcrest `containsInAnyOrder`. Faire échouer les deux versions et comparer les messages. Commencer par écrire la description du test.

Réponse :

Question 18. Écrire un test **orienté interaction** sur le même scénario : vérifier avec `verify` que `AllVersionsResolver` interroge le registre pour chacune des trois coordonnées, et **une seule fois** pour chacune. Quelle propriété du résolveur ce second test capture-t-il, que le premier ne capture pas ? Commencer par écrire la description du test.

Réponse :

Question 19. Le cas d'un cycle figure-t-il dans votre task list ? Sinon, l'y ajouter. Ajouter ensuite un cycle **dans les données du registre bouchonné** (`lib-c` dépend de `lib-a`). Écrire d'abord le test et constater l'échec (que se passe-t-il ?), puis corriger `AllVersionsResolver`. Le test d'interaction de la question 18 vous aide-t-il à diagnostiquer plus vite que le test d'état ?

Réponse :

Question 20. Un test d'interaction est plus précis mais aussi plus *fragile* : il se casse lors de refactorings qui ne changent pourtant pas le comportement observable. Donner un exemple concret sur `AllVersionsResolver`. Quelle règle en tirer sur l'usage de `verify` ?

Réponse :

Test unitaire de `BuildTool`

Question 21. Tester unitairement `BuildTool`, avec des doublures de `IPomParser` et de `IResolver`. Sa responsabilité propre est d'enchaîner les deux : le test doit vérifier que `BuildTool` transmet au résolveur exactement les dépendances issues du parseur. Commencer par écrire la description du test.

Partie 4 — Tests d'intégration

Les tests unitaires garantissent que chaque composant fonctionne isolément. Ils ne disent **rien** sur la qualité des **interfaces de communication** entre composants — objet des tests d'intégration.

Exemple : si vous avez choisi en question 14.1 que `parse` renvoie `null` pour un fichier vide, alors que le test unitaire de `BuildTool` (question 21) utilise un bouchon de `IPomParser` qui renvoie toujours un `Project`, les deux tests unitaires passent, mais l'assemblage lève une `NullPointerException` : chaque composant respecte sa propre hypothèse, et personne n'a vérifié l'interface commune.

Pilotes et bouchons

Pour tester un composant en cours d'intégration, il faut :

- un **pilote** (*driver*) qui joue le rôle des composants appelants non encore intégrés : il fournit les jeux d'essai au composant à tester et en recueille les résultats (C2, diapositives 24-25) ;
- un ou plusieurs **bouchons** (*souches*, *stubs*) qui jouent le rôle des composants appelés non encore intégrés (C2, diapositives 24-25).

On suit la démarche incrémentale du cours (C2, diapositive 40, « Exemple d'intégration incrémentale ») : à chaque étape on **ajoute** un ou plusieurs composants à ceux déjà intégrés, et l'étape se caractérise par les **interfaces** qu'elle permet de tester.

Question 22. Reprendre le graphe d'appel de `minibuild`. Pour chaque composant, indiquer qui sont ses appelants et ses appelés. Quels composants n'ont aucun appelant ? Aucun appelé ?

Réponse :

Composant	Appelants	Appelés
<code>BuildTool</code>		
<code>LineBasedPomParser</code>		
<code>AllVersionsResolver</code>		
<code>StorageBasedRegistry</code>		
<code>InMemoryStorage</code>		
<code>BufferedReader</code>		

Question 23. Pour chaque composant, calculer sa *hauteur* (longueur du plus long chemin vers un composant sans appelé) et sa *profondeur* (longueur du plus long chemin depuis un composant sans appelant). Que remarquez-vous ?

Réponse :

Composant	Hauteur	Profondeur
<code>BuildTool</code>		
<code>LineBasedPomParser</code>		
<code>AllVersionsResolver</code>		
<code>StorageBasedRegistry</code>		
<code>InMemoryStorage</code>		
<code>BufferedReader</code>		

Stratégie incrémentale descendante

On intègre d'abord les composants de plus haut niveau, en simulant ceux de plus bas niveau.

Question 24. Proposer un ordre d'intégration descendant pour `minibuild`. Pour chaque étape, indiquer les composants ajoutés, les interfaces testées, les bouchons nécessaires et les pilotes nécessaires.

Réponse :

Étape	Composants ajoutés	Interfaces testées	Bouchons	Pilotes
1				
2				
...				

Question 25. Implémenter l'étape 1 de votre ordre descendant : les composants ajoutés sont réels, les autres sont simulés. Le test porte sur l'**interface** entre les composants ajoutés, pas sur leur logique interne, déjà couverte par les tests unitaires de la partie 3. En quoi ce test diffère-t-il du test unitaire de `BuildTool` (question 21) ?

Réponse :

Question 26. Poursuivre l'intégration descendante jusqu'à ce que tous les composants soient réels. À chaque étape, conserver les tests des étapes précédentes : ils doivent continuer à passer.

Question 27. Le cours (C2, diapositive 35) indique que la stratégie descendante détecte tôt les défauts de structure, mais que la logique des bouchons doit être élaborée. Illustrer ces deux points sur votre intégration de `minibuild`.

Réponse :

Stratégie incrémentale ascendante

On intègre d'abord les composants de plus bas niveau, en simulant les appelants par des pilotes.

Question 28. Proposer un ordre d'intégration ascendant pour `minibuild`, sous la même forme qu'en question 24.

Réponse :

Étape	Composants ajoutés	Interfaces testées	Bouchons	Pilotes
1				
2				
...				

Question 29. Implémenter l'étape 1 de votre ordre ascendant, avec un pilote qui fournit les jeux d'essai.

Question 30. Comparer les deux stratégies sur ce système : nombre de pilotes, nombre de bouchons, moment où chaque défaut serait détecté. Laquelle recommanderiez-vous ici, et pourquoi ?

Réponse :

Critère	Descendante	Ascendante
Nombre de pilotes		
Nombre de bouchons		
Défauts de structure détectés		
Interfaces bas niveau testées		

Question 31. Une troisième voie, présentée dans le cours (C2, diapositive 32), consiste à tout assembler d'un coup (*big bang*). Quel est son coût en pilotes et bouchons ? Pourquoi est-elle malgré tout déconseillée ?

Réponse :

Partie 5 — Tests de validation

Les tests de validation exercent le système **complet**, sans aucune doublure, à travers ses interfaces externes, et confrontent son comportement à la spécification. Ce sont des tests « boîte noire » au niveau système.

Question 32. Écrire un test de validation de bout en bout :

- un vrai fichier de build sur disque, dans les ressources de test ;
- un vrai `BufferedReader` sur ce fichier ;
- un registre réel, pré-alimenté avec les artefacts et leurs dépendances ;
- vérification de l'ensemble des dépendances résolues avec Hamcrest.

Question 33. À la fin de l'intégration descendante (question 26), tous les composants sont réels. En quoi votre dernier test d'intégration diffère-t-il de ce test de validation ?

Réponse :

Question 34. Ajouter deux tests de validation supplémentaires : un fichier de build référençant une dépendance absente du registre, et un fichier syntaxiquement invalide. Que doit faire le système dans chaque cas ? La spécification du sujet répond-elle à cette question ?

Réponse :

Question 35. Un test de validation qui échoue vous indique-t-il quel composant est fautif ? Comparer avec un test unitaire et un test d'intégration qui échouent. Qu'en déduisez-vous sur la complémentarité des trois niveaux ?

Réponse :

Question 36. Le cours (C1, diapositive 26, « Qui fait quoi ? ») recommande que les tests de validation soient conduits par une équipe indépendante. Pourquoi ? Que recommande-t-il pour les tests d'intégration et pour les tests unitaires, et pourquoi cette différence ? Dans ce TD, vous avez écrit seul les trois niveaux de test : qu'est-ce que cela change ?

Réponse :

Partie 6 — Couverture

JaCoCo mesure la couverture du code par les tests.

```
./gradlew test jacocoTestReport
```

Question 37. Avant de lancer la mesure : quel taux de couverture attendez-vous, sachant que l'essentiel du code a été écrit en TDD ? Justifier.

Réponse :

Question 38. Lancer la mesure et ouvrir le rapport HTML. Comparer la couverture obtenue avec votre prédiction. Identifier les portions non couvertes et compléter les tests si nécessaire.

Réponse :

Question 39. Comparer la couverture apportée par chaque niveau de test en exécutant séparément les tests unitaires, d'intégration et de validation (piste : étiqueter les tests avec l'annotation JUnit `@Tag` et filtrer dans Gradle — voir les documentations de JUnit et de Gradle).

Réponse :

Question 40. Un taux de couverture élevé garantit-il l'absence de défauts ? JaCoCo mesure notamment la couverture des instructions et celle des branches : à quels critères du cours (C4, diapositive 19) correspondent-elles ? Construire sur `minibuild` un défaut réel non détecté malgré 100 % de couverture des instructions. Résiste-t-il à 100 % de couverture des branches ?

Réponse :

Partie 7 — Bilan

Question 41. Comparer les trois niveaux de test pratiqués dans ce TD, en illustrant chaque case par un exemple tiré de votre travail sur `minibuild`.

Réponse :

Niveau	Objet du test	Doublures utilisées	Défauts détectés	Localisation du défaut
Unitaire				
Intégration				
Validation				

Question 42. Discuter des avantages et des inconvénients du TDD par rapport à vos pratiques de développement habituelles. Sur quel type de composant de `minibuild` le TDD vous a-t-il le plus aidé ? Le moins ?

Réponse :
